

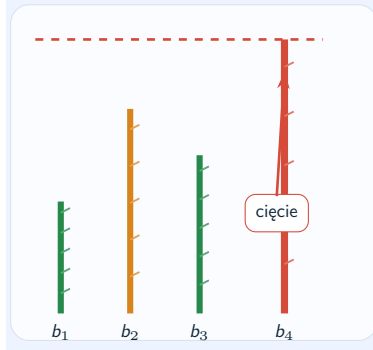
Bamboo Garden Trimming

Jak utrzymać wiecznie rosnący ogród pod kontrolą?

Michał Bober Piotr Komisarczyk Igor Staręga

Gry Kombinatoryczne – Politechnika Warszawska

Cel projektu: przełożyć wyniki teoretyczne o BGT na czytelny model, porównanie algorytmów i działającą symulację w Pythonie.



Problem w jednym obrazku

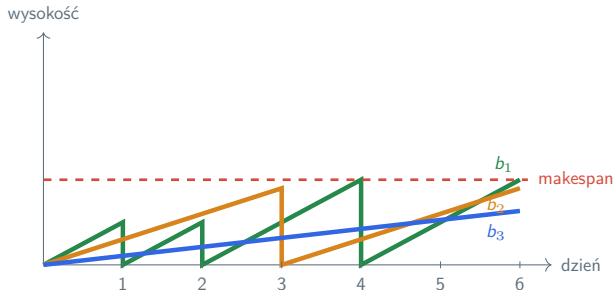
Każdy dzień ma dwa kroki:

1. wszystkie bambusy rosną,
2. robot może skosić jeden z nich.

współczynniki wzrostu

decyzje online

harmonogram wieczysty



Model formalny

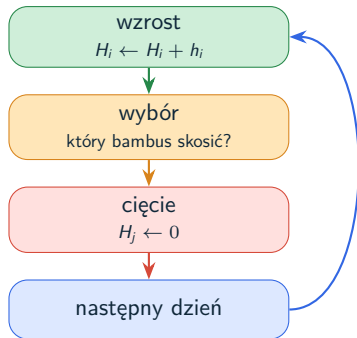
Instancja: n bambusów z szybkościami $h_1, \dots, h_n > 0$. Normalizujemy:

$$\sum_{i=1}^n h_i = 1.$$

Wysokość przed cięciem:

$$H_i(d) = (d - d')h_i.$$

d' to ostatni dzień cięcia bambusa b_i .



Minimalizujemy

$$\mathcal{M} = \max_{i,d} H_i(d).$$

Granice: optimum jest blisko 2

Dolna granica

$$\mathcal{M} \geq 1$$

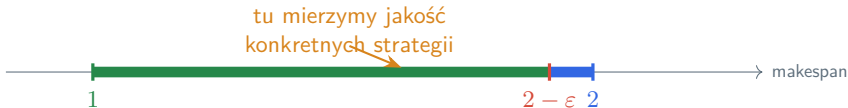
ogród rośnie łącznie o 1
dziennie

Prawie 2 bywa konieczne

$$h_1 = 1 - \varepsilon, h_2 = \varepsilon$$

2 zawsze wystarcza

przez redukcję do Pin-
wheel Scheduling



Algorytm 1: Reduce-Max

Reguła:

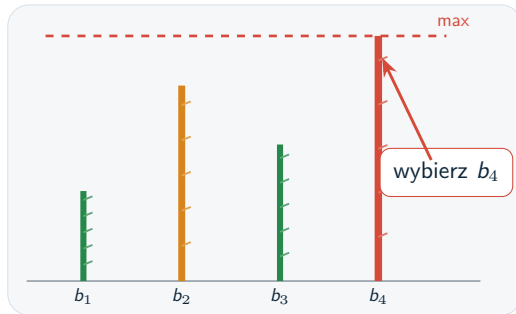
każdego dnia skosimy bambus o największej aktualnej wysokości:

$$j = \arg \max_i H_i(d).$$

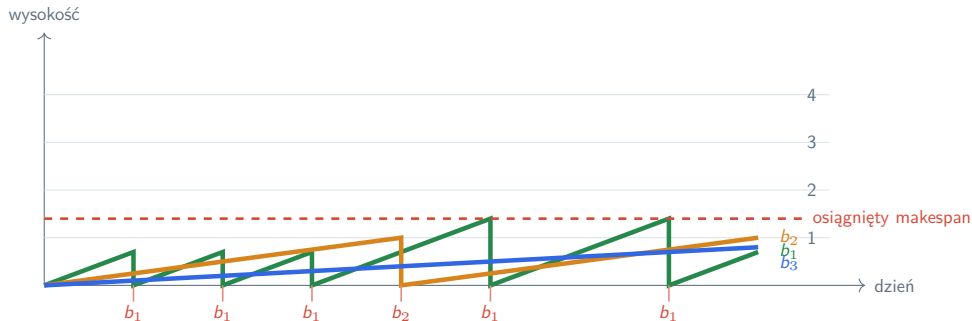
Wynik teoretyczny:

Reduce-Max $\Rightarrow \mathcal{M} \leq 9$.

Hipoteza eksperymentalna: granica 2.



Wizualizacja Reduce-Max: co dzieje się w czasie?



Czerwone podpisy pod osią pokazują, który bambus został skoszony danego dnia.

Algorytm 2: Reduce-Fastest(x)

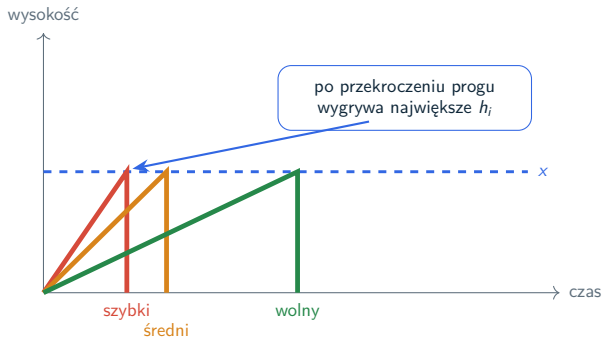
Reguła:

patrzemy tylko na bambusy, które przekroczyły próg x , a potem wybieramy najszybciej rosnący:

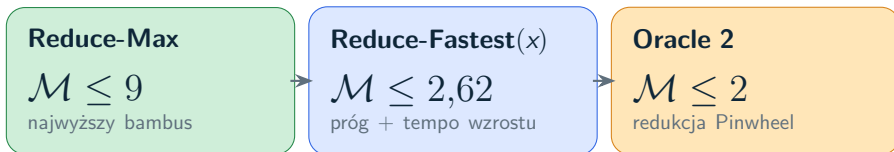
$$j = \arg \max_{i: H_i(d) \geq x} h_i.$$

Najlepszy próg z analizy:

$$x = 1 + \frac{1}{\sqrt{5}}, \quad \mathcal{M} \leq \frac{3 + \sqrt{5}}{2} \approx 2,618.$$



Porównanie strategii bez ściany tabeli



Interpretacja: prosta reguła może być świetna eksperymentalnie, ale dowód najlepszej gwarancji wymaga innej struktury.

Redukcja do Pinwheel Scheduling

Pinwheel Scheduling

Okno długości f_i zawsze zawiera zadanie i . Gęstość:

$$\rho = \sum_i \frac{1}{f_i}.$$

Jeśli $\rho \leq 1$ i f_i są potęgami 2, harmonogram istnieje.

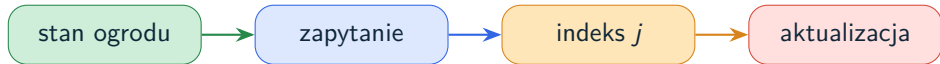
$$f_i = 2^{\lceil \log_2(2/h_i) \rceil}$$
$$\rho = \sum_i \frac{1}{f_i} \leq \sum_i h_i = 1$$



harmonogram okresowy

Wniosek: z Pinwheel dostajemy BGT z makespanem co najwyżej 2.

Trimming Oracle: strategia kontra szybkie zapytanie



Reduce-Fastest(x)
priority search tree
 $O(\log n)$

Reduce-Max
górną otoczką prostych
 $O(\log^2 n)$

Makespan 2
dekompozycja regularna
 $O(\log n)$

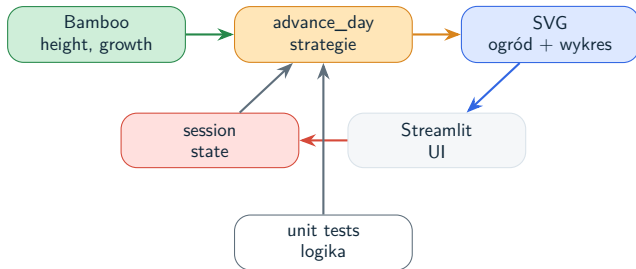
Co zrobiliśmy w projekcie?

Implementacja w Pythonie

model bambusa, operacje grow/cut, wspólny krok symulacji i trzy strategie wyboru.

Interfejs Streamlit

tryby Sandbox, Compare i Game oraz wizualizacje SVG.



Wizualizacja nie jest dodatkiem: pozwala zobaczyć, jak decyzje algorytmu zmieniają historię makespanu.

Tryby aplikacji

Sandbox

parametry ogrodu,
strategia i krok
symulacji

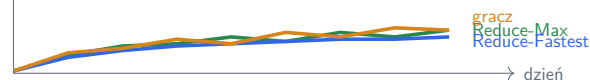
Compare

Reduce-Max,
Reduce-Fastest(x)
i oracle działają na
tym samym wejściu

Game

użytkownik tnie
bambusy i gra
przeciw botowi

makespan



Otwarte problemy

Reduce-Max vs 2

Czy Reduce-Max ma
wynik 2?

Reduce-Fastest⁽¹⁾

Czy próg 1 wystarcza
do granicy 2?

Praktyczne oracle

Jak proste kopce i
drzewa wypadają w
testach?

Aplikacja jest dobrym miejscem do eksperymentów, które mogą podpowiedzieć hipotezy dla mocniejszych dowodów.

Najważniejsze wnioski

Prosty model trudna analiza

Jeden robot. Jedno
cięcie. Nietrywialna
gra.

Granica 2 jest kluczowa

Redukcje dają granicę
2.

Symulacja robi różnicę

Te same dane tworzą
różne historie cięć.

Dziękujemy

Bibliografia



D. Bilò, L. Gualà, S. Leucci, G. Proietti, G. Scornavacca, *Cutting bamboo down to size*, Theoretical Computer Science 909, 2022.



L. Gąsieniec, R. Klasing, C. Levcopoulos, A. Lingas, J. Min, T. Radzik, *Bamboo garden trimming problem*, SOFSEM 2017.



R. Holte, A. Mok, L. Rosier, I. Tulchinsky, D. Varvel, *The pinwheel: a real-time scheduling problem*, HICSS 1989.